

Introducción a Scala

Procesamiento de Datos

Miguel Ángel Fernández Díaz



¿Qué es Scala ?

Scala = Orientado a objetos + Funcional
Lenguaje de alto nivel sobre la JVM
Tipado estático
Multiparadigma
Interoperable con Java

JVM

```
object MyClass extends App {  
  println("Hello world")  
}
```

MyClass.java

scalac



javac



MyClass.class



JVM bytecode

JVM

```
public final class MyClass {  
  public static void main(java.lang.String[]);  
  Code:  
    0: getstatic      #16          // Field MyClass$.MODULE$:LMyClass$;  
    3: aload_0  
    4: invokevirtual #18          // Method MyClass$.main:([Ljava/lang/String;)V  
    7: return
```

¿Por qué Scala?

- Código conciso
- Funciones como ciudadanos de primera clase
- Inmutabilidad
- Pattern Matching

Comparación rápida con Java

- Menos boilerplate
- Inferencia de tipos
- Case class
- Funciones lambda más expresivas

Características

- Lenguaje de alto nivel sobre la JVM, centrado en FP aunque permite P. Imperativa
- Estáticamente tipado
- Compatibilidad bidireccional con librerías en Java
- Conversión no directa con:
 - Tipos nativos
 - Varargs
 - Collections
 - Funciones de orden superior (Directa desde Scala 2.12)
 - Java 8 streams (Directa desde Scala 2.12)

Compatibilidad con Java

```
import java.time.LocalDateTime
...
def getNextDay() : LocalDateTime = {
  val currentTime : LocalDateTime = LocalDateTime.now()
  val nextDate = currentTime plusDays 1
  nextDate
}
```

Despliegue de imagen

```
docker run -it --rm -v "$PWD:/workspace" scala212:1.0
root@9cd6557c502c:/workspace# scala
Welcome to Scala 2.12.20 (OpenJDK 64-Bit Server VM, Java 11.0.29).
Type in expressions for evaluation. Or try :help.

scala> :q
```


Basics

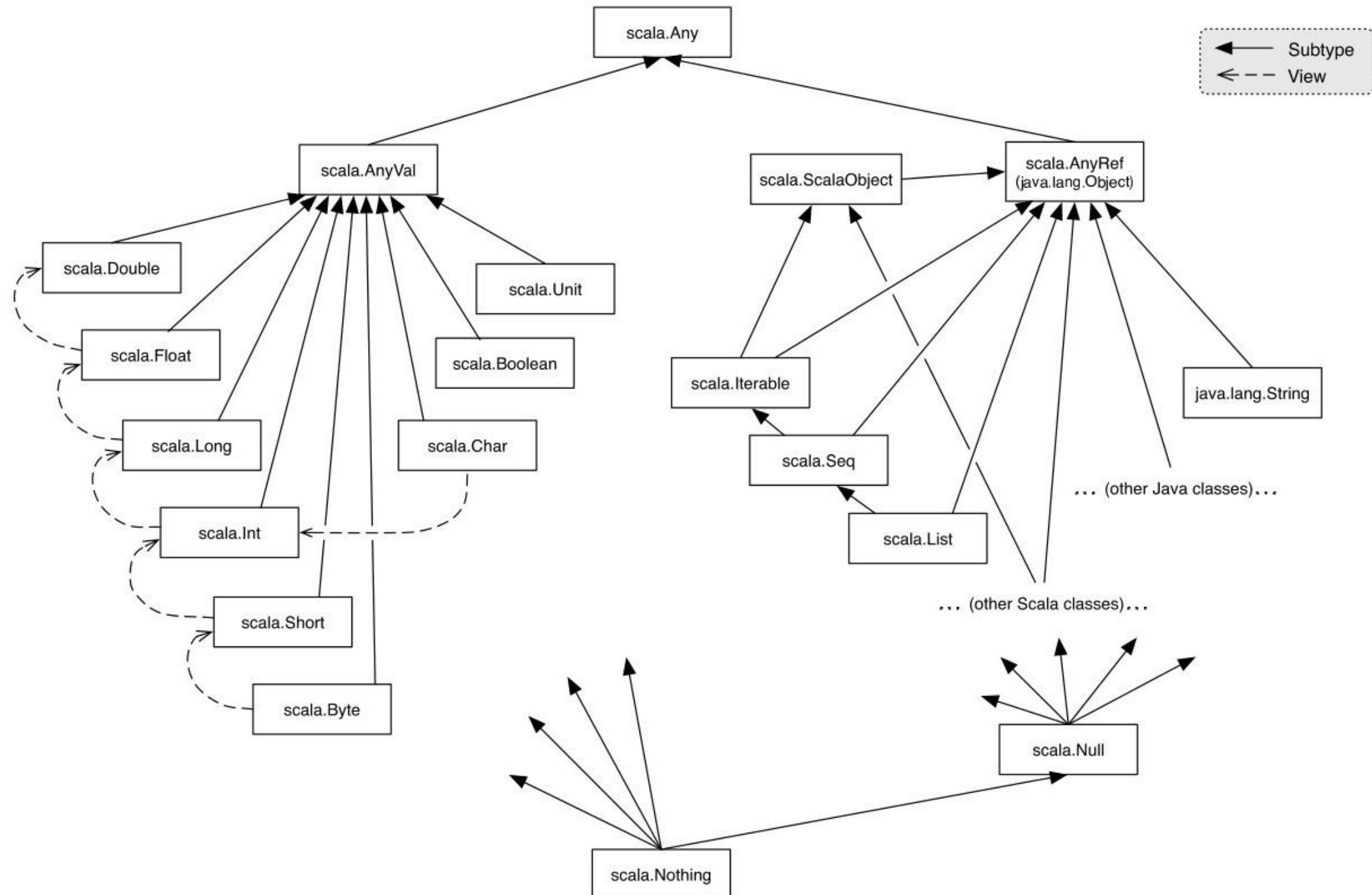
No ;

No NULL

Immutable

Classes

Type hierarchy: TODO es un objeto, hasta la nada



Hello World

```
object HelloWorld {  
  def main(args: Array[String]) {  
    println("Hello world from main!")  
  }  
}
```

```
$ scalac HelloWorld.scala  
$ scala HelloWorld
```

```
object HelloWorld extends App {  
  println("Hello world from implicit main!")  
}
```

Classes & Companion objects

```
class MiClase(nombre: String) { // Clase
  private def saludar(): Unit = {
    println(s"Hola, $nombre")
  }
  def realizarSaludo(): Unit = {
    MiClase.saludoPersonalizado(nombre) // Llamada a un método privado del companion object
  }
}

object MiClase { // Companion Object
  def saludoPersonalizado(nombre: String): Unit = { // Método definido en el companion object
    println(s"Bienvenido, $nombre")
  }
  val PI = 3.1416 // Ejemplo de un valor compartido
}

object Ejemplo { // Uso de la clase y el companion object
  def main(args: Array[String]): Unit = {
    val instancia = new MiClase("Juan")
    instancia.realizarSaludo() // Llamada a un método de la clase
    MiClase.saludoPersonalizado("María") // Llamada a un método del companion object
    println(s"El valor de PI es: ${MiClase.PI}") // Acceso a un valor compartido del companion object
  }
}
```


Declaración de variables

Declaración de variables

- Declaración de valores inmutables con val
- Declaración de variables mutables con var

```
var mutableVariable : String = "InitialValue"
```

```
val immutableVariable : String = "ImmutableValue"
```

Tipado con inferencia

- `val x = 10`
- `val y: Int = 20`

Options:

- Some(valor)
- None
- Evita NullPointerException

Always have options

- Declaración de variables inmutables con val
- Declaración de variables mutables con var

```
scala> var c : Option[MyClass] = None  
c: Option[MyClass] = None
```

```
scala> c.isDefined  
res17: Boolean = false
```

```
scala> c = Some(new MyClass)  
c: Option[MyClass] = Some(MyClass())
```

```
scala> c.isDefined  
res18: Boolean = true
```

getOrElse

- `opt.getOrElse(valorPorDefecto)`

Ejemplo: Option

- `val opt: Option[Int] = Some(10)`
- `opt.getOrElse(0)`

Funciones

Definición de funciones

- `def doble(x: Int): Int = x * 2`

Funciones como valores

- `val f = (x: Int) => x * 2`

Funciones como parámetro

- `def aplicar(x: Int, f: Int => Int) = f(x)`

Métodos y funciones

```
def functionName ([list of parameters]) : [return type] = {  
    function body  
    [return expr]  
}
```

```
override def drop(n: Int): List[A] = {  
    var these = this  
    var count = n  
    while (!these.isEmpty && count > 0) {  
        these = these.tail  
        count -= 1  
    }  
    these  
}
```

Métodos y funciones

```
def print(x: Any) = Console.print(x)
def println() = Console.println()
def println(x: Any) = Console.println(x)
def printf(text: String, xs: Any*) = Console.print(
  text.format(xs: _*))
```

Funciones como parámetro

```
final override def foreach[U](f: A => U) : Unit = {  
  var these = this  
  while (!these.isEmpty) {  
    f(these.head)  
    these = these.tail  
  }  
}
```

Case classes

Case class, el tipo producto

- Clase estándar
- Los parámetros del constructor son valores
- Permite pattern matching
- Incluye factorías
- Es serializable y añade copy
- Es un producto (extiende Product)

```
abstract class Expr
case class Var(name: String) extends Expr
case class Number(num: Double) extends Expr
case class UnOp(operator: String, arg: Expr) extends Expr
case class BinOp(operator: String,
  left: Expr, right: Expr) extends Expr
```

case class

- Clases inmutables
- Constructor automático
- copy, equals, toString

Ejemplo: case class

- `case class Alumno(nombre:String, nota:Int)`
- `val a = Alumno("Ana",8)`
- `println(a.nombre)`

Pattern matching

```
def simplifyTop(expr: Expr): Expr = expr match {  
  case UnOp("-", UnOp("-", e)) => e // Double negation  
  case BinOp("+", e, Number(0)) => e // Adding zero  
  case BinOp("*", e, Number(1)) => e // Multiplying by one  
  case _ => expr  
}
```

Pattern Matching

- Sustituye a if/else y switch
- Muy potente

Ejemplo: Pattern Matching

- `def evaluar(x:Int) = x match {`
- `case 0 => "cero"`
- `case n if n>0 => "positivo"`
- `case _ => "negativo"`
- `}`

Traits:

- Similares a interfaces
- Permiten implementación
- Soportan herencia múltiple

Traits

```
trait Similarity {  
  def isSimilar(x: Any): Boolean  
  def isNotSimilar(x: Any): Boolean = !isSimilar(x)  
}
```

```
class Point(xc: Int, yc: Int) extends Similarity {  
  var x: Int = xc  
  var y: Int = yc  
  def isSimilar(obj: Any) =  
    obj.isInstanceOf[Point] &&  
    obj.asInstanceOf[Point].x == x  
}
```

Traits

```
object StringIteratorTest {  
  def main(args: Array[String]) {  
    class Iter extends StringIterator(args(0))  
    with RichIterator  
    val iter = new Iter  
    iter foreach println  
  }  
}
```

- Generics

```
class Stack[T]
```

Generics

```
1 class Stack[T] {  
2   var elems: List[T] = Nil  
3   def push(x: T) { elems = x :: elems }  
4   def top: T = elems.head  
5   def pop() { elems = elems.tail }  
6 }
```


Ejemplo: Genéricos

```
class Caja[T](val x:T)  
val c = new Caja[Int](5)
```

- Programación funcional

Funciones puras

Inmutabilidad

Transformaciones de datos

FP: Programar, ahora, ¡es fácil!

- No hay cambios inesperados
- Nuestros bugs están encerrados
- ¿Proteger nuestros accesos concurrentes? ¿Para qué si nada puede cambiar?
- No hay que razonar sobre el paso por valor o por referencia
- Los programas parecen fórmulas. Es fácil verificar que son correctos

Colecciones con funciones de orden superior

Colecciones básicas

- List, Set, Map, Seq

map / filter / reduce

- `list.map(_*2)`
- `list.filter(_>3)`
- `list.reduce(_+_)`

flatMap

- Convierte N elementos en M elementos
- Muy usado con listas y texto

foldLeft

- Acumulador explícito
- Útil para cálculos progresivos

Map

```
final def flatMap[B](f: A => List[B]): List[B]
```

```
final def map[B](f: A => B): List[B]
```

```
scala> val myList = List(1,2,3,4,5,6)  
myList: List[Int] = List(1, 2, 3, 4, 5, 6)
```

```
scala> myList.map(value => value * 2)  
res1: List[Int] = List(2, 4, 6, 8, 10, 12)
```

Filter

```
def filter(p: (A) => Boolean): List[A]
```

```
scala> val myList = List(1,2,3,4,5,6)  
myList: List[Int] = List(1, 2, 3, 4, 5, 6)
```

```
scala> myList.filter(_ > 3)  
res2: List[Int] = List(4, 5, 6)
```

Reduce

```
def reduce[A1 >: A](op: (A1, A1) => A1): A1
```

```
scala> val myList = List(1,2,3,4,5,6)  
myList: List[Int] = List(1, 2, 3, 4, 5, 6)
```

```
scala> myList.reduce((a,b) => a + b)  
res3: Int = 21
```

Fold

```
def fold[A1 >: A](z: A1)(op: (A1, A1) => A1): A1
```

```
scala> myList.fold(0)((acc, cur) => {acc + cur})  
res16: Int = 21
```

FoldLeft

```
def foldLeft[B](z: B)(f: (B, A) => B): B
```

```
scala> myList.foldLeft(Map.empty[Int, Int])(  
(acc, cur) => {acc + (cur -> cur *2)}  
)  
res5: scala.collection.immutable.Map[Int,Int] =  
Map(5 -> 10, 1 -> 2, 6 -> 12,  
2 -> 4, 3 -> 6, 4 -> 8)
```

Ejemplo: map / filter / reduce

- `val nums = List(1,2,3,4,5)`
- `nums.map(_*2)`
- `nums.filter(_%2==0)`
- `nums.reduce(_+_)`

For comprehension:

Sintaxis expresiva para combinaciones y filtros

For

- Diferencias semánticas con Java

```
for(x <- c1; y <- c2; z <- c3) {...}
```



```
c1.foreach(x => c2.foreach(y => c3.foreach(z => {...})))
```

```
for(x <- c1; y <- c2; z <- c3) yield {...}
```



```
c1.flatMap(x => c2.flatMap(y => c3.map(z => {...})))
```


For-yield

```
object ComprehensionTest1 extends Application {  
  def even(from: Int, to: Int): List[Int] =  
    for (i <- List.range(from, to) if i % 2 == 0) yield i  
  Console.println(even(0, 20))  
}
```

Try-catch

Try / Success / Failure

- Manejo funcional de excepciones

Try-catch

```
try {  
    // ...  
} catch {  
    case ioe: IOException => ...  
    case e: Exception => ...  
} finally {  
    ...  
}
```

Try

```
def divide: Try[Int] = {  
  val dividend = Try(Console.readLine("Int dividend:\n").toInt)  
  val divisor = Try(Console.readLine("Int divisor:\n").toInt)  
  val problem = dividend.flatMap(x => divisor.map(y => x/y))  
  problem match {  
    case Success(v) =>  
      println("Result: " + dividend.get + "/"  
        + divisor.get + " is: " + v)  
      Success(v)  
    case Failure(e) =>  
      println("Division error. Try again!")  
      println("Info from the exception: " + e.getMessage)  
      divide  
  }  
}
```

Otros elementos

- Implicits→
- Modos de herencia:
 - Simple
 - Multiple
 - Traits mixins
 - Abstracta
- Sbt vs Maven
- [Rendimiento de las colecciones](#)

```
// Definición de la clase Persona
case class Persona(nombre: String)

// Definición de una clase que representa saludos en diferentes idiomas
class SaludosEnDiferentesIdiomas {
  def saludarEnIngles(persona: Persona): String = s"Hello, ${persona.nombre}!"
  def saludarEnEspanol(persona: Persona): String = s"Hola, ${persona.nombre}!"
}

// Definición de un objeto con una instancia de SaludosEnDiferentesIdiomas como implícito
object SaludosImplicits {
  implicit val saludos = new SaludosEnDiferentesIdiomas
}

// Función de saludo que utiliza el parámetro implícito
def saludar(persona: Persona)(implicit saludos: SaludosEnDiferentesIdiomas): String = {
  saludos.saludarEnIngles(persona) // Usa el saludo en inglés de manera implícita
}

// Uso de la función de saludo
import SaludosImplicits.saludos // Importa el implícito
val persona = Persona("John")
val saludo = saludar(persona)
println(saludo) // Imprime: Hello, John!
```

Buenas prácticas

- Evitar var
- Evitar bucles
- Pensar en transformaciones

Resumen

- Scala combina OO y funcional
- Código más compacto que Java
- Base sólida para sistemas modernos

Ejercicio 1 – Colecciones

- Dada la lista:
- `val l = List(3,6,1,9,2)`
- 1) Obtener solo los pares
- 2) Multiplicarlos por 10
- 3) Calcular la suma final

Ejercicio 2 – case class

- Crear una case class Persona(nombre, edad)
- Crear una lista de personas
- Filtrar mayores de 18

Ejercicio 3 – Pattern Matching

- Implementar función que clasifique una nota:
- <5 Suspenso
- 5-6 Aprobado
- 7-8 Notable
- 9-10 Sobresaliente

Ejercicio 4 – Option

- Crear función que devuelva `Option[Int]`
- Usar `getOrElse` para valor por defecto

Ejercicio 5 – Genéricos

- Crear clase Stack[T]
- push, pop, top

Mini proyecto

- Lista de alumnos:
- - Filtrar aprobados
- - Calcular media
- - Mostrar nombres en mayúsculas

Ejercicios propuestos

Ejercicio 1

- Crear una función que reciba como parámetro una lista y una función, y realice las siguientes operaciones
 - Invertir el orden de la lista
 - Aplicar la función a cada elemento de la lista
 - Devolver un mapa asociando cada valor de la lista con el resultado de la función.
 - La función debe utilizar **foldLeft**

```
val l = List(1,2,3,4,5)
val r1 = invertAndMap(l, transformToString)
println(r1)
> Map(5 -> string: 5, 1 -> string: 1, 2 -> string: 2,
      3 -> string: 3, 4 -> string: 4)
val r2 = invertAndMap(l, square)
println(r2)
> Map(5 -> 25, 1 -> 1, 2 -> 4, 3 -> 9, 4 -> 16)
```


Ejercicio 2

- Defina una case class Persona con atributos nombre, edad, y email
- Genere automáticamente 10 instancias de Persona
 - Pueden tener nombres consecutivos
- Genere una función que agrupe todas las personas con una misma edad en una estructura tipo: Map[Int, List[Person]]
- Ofrezca dos implementaciones (al menos una funcional)

Ejercicio 3

- Lea un fichero de texto donde cada línea contiene la información de una persona utilizando un separador especificado como parámetro
- Codificar una función que transforme una línea de fichero en una Persona
- Filtre aquellas personas mayores que 18 y cuyo correo termine en extensión .com
- Escriba el resultado en un fichero de texto con el mismo formato del anterior.